What is Feature,Scaling

Male Hight In Feet	Female Hight In CM	Life Span In Year
8.0	150	30
8.5	165	40
7.9	170	36
8.2	140	41

What is Feature Scalling?

Feature scalling is a method to scale numeric feature in the same scale or range(like -1 to 1, 0 to 1)

It is also called normalization

Apply feature scalling on independent variable

Fit feature scalling with train data and transform on train and test data

Why Feature scalling?

The scale of row feature is different according to its units

Machine learning algorithms can’t understand feature units, understand only numbers.

Ex; if hight 150cm and 8.2 feet

Ml algorithms understand number 150>8.2

Type of feature scalling

- Min max scaler
- Standard scaler
- Max abs scaler
- Robust scaler
- Quantile trandformer scaler
- Power transformer scaler
- Unit vector scaler

Standardization &normalization

What is Standardization?

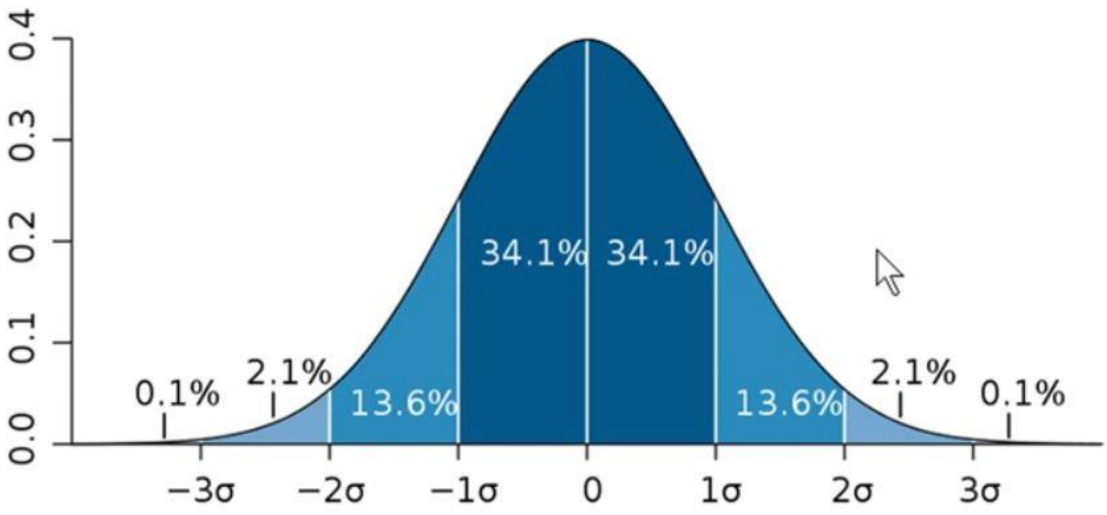
Standardization rescale the feature such as mean(μ)=0 and standard deviation (σ)=1

The result of standardization is z called as Z-score normalization

$$z = \frac{x - \mu}{\sigma}$$

If data follow normal distribution use standard deviation

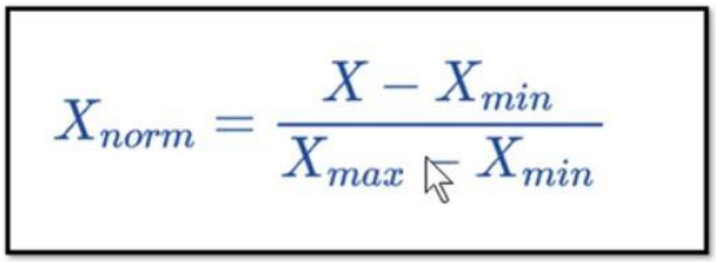
If the original distribution is normal then standard distribution will be normal



What is normalization

Normlization rescale the feature in fixed range between 0 to 1

Normalization also called as Min-Max Scalling



$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

If data doesn't follow normal distribution

Now import standrd Scaller & min max scaler to your project

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
```

now load dataset

```
df=sns.load_dataset('titanic')
df.head()
```

feature scalling apply after cleaning all the data & apply only on numeric data

now we will use some feature for demostration

```
df2=df[['survived','pclass','age','parch']]
df2.head()
```

in this dataset survived is a targeted variable

now we will clean the data set … we fill the mean value in dataset

```
df3=df2.fillna(df2.mean())
df3.head()
```

now we need to convert data into metric or vector

those independent variable we need to convert into metric

and those dependent variable consider vector

now we need to x, and y variable .

survived in dependent variable we need to drop this column

```
X=df3.drop('survived',axis=1)
y=df3['survived']
print('x shape=',X.shape)
print('y shape=',y.shape)
```

now we need to import model selection library of machine learning to train and test dataset

```
from sklearn.model_selection import train_test_split
```

use train_test-split() will give 4 dataset x-train,y-train,x-test,y-test

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=51)
print('X-train shape=',X_train.shape)
print('X-test shape=',X_test.shape)
print('y-train shape=',y_train.shape)
print('t-test shape=',y_test.shape)
```

now apply feature scalling to use standard scaler

```
sc=StandardScaler()
sc.fit(X_train)
```

if you want value of standard scaller

```
sc.mean_
```

if you want to check standard deviation

```
sc.scale_
```

now find want to check detail use

```
X_train.describe()
```

Now transform dataset

```
X_train_sc=sc.transform(X_train)
X_test_sc=sc.transform(X_test)
```

Now convert data into dataframe

```
X_train_df=pd.DataFrame(X_train_sc,columns=['pclass','age','parch'])
X_test_df=pd.DataFrame(X_train_sc,columns=['pclass','age','parch'])
```

Now find mean value and standard deviation

```
X_train_df.describe().round(2)
```

Now use minmaxscaller function for standardization

```
mmc=MinMaxScaler()
mmc.fit(X_train)
```

now transform this dataset

```
X_train_mmc=mmc.transform(X_train)
X_test_mmc=mmc.transform(X_test)
```

Now create dataframe

```
X_train_mmc_df=pd.DataFrame(X_train_mmc,columns=['pclass','age','parch'])
X_test_mmc_df=pd.DataFrame(X_train_mmc,columns=['pclass','age','parch'])
```

Now find describe the dataset

```
X_train_mmc_df.describe().round(2)
```

Now find the pairplot of original dataset

```
sns.pairplot(X_train)
```

now find the plot of new dataset

```
sns.pairplot(X_train_df)
```

clearly see that there is no huge difference

now clean dataset view

```
sns.pairplot(X_train_mmc_df)
```